UNIVERSIDAD NACIONAL DEL ALTIPLANO PUNO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL INGENIERÍA DE SISTEMAS



CURSO:

Programación Orientada a Objetos-II

TRABAJO:

Abstracción y Encapsulamiento: Invariantes, propiedades y Diseño por Responsabilidades

PRESENTADO POR:

Melvin Ronald Choque Chaiña

DOCENTE:

Mg. Aldo Hernan Zanabria Galvez

FECHA:

13/09/2026

PUNO-PERÚ

Contenido

1. Objetivo General 3

1.1. Objetivos Específicos 3

2. Marco Teórico 3

2.1. Invariantes de Clase – El Contrato Interno 3

2.2. Name Mangling en Python – _atributo 4

2.3. Propiedades Calculadas (read-only) en Python 5

2.4. Diseño de responsabilidades — ¿A quién pertenece este dato? 5

3. Materiales 6

4. Instrucciones Previas 6

5. Desarrollo de actividad 7

5.1. Actividad 1 — @property avanzado — Invariantes con setters validados (Python) 7

5.2. Actividad 2 — Propiedades calculadas y encadenamiento (Python) 9

5.3. Actividad 3 — Encapsulamiento en C++17 — const correctness y setters validados 12

5.4. Actividad 4 — Comparación Python vs C++ — tabla de equivalencias de encapsulamiento 13

6. Diagramas UML 15

7. Preguntas de Reflexión 15

1. Objetivo General

Diseñar e implementar clases de encapsulamiento superficial (solo ocultar datos) y encapsulamiento estricto mediante invariantes de clase, propiedades con validación (Python) y getters/setters con control de invariantes (C++), aplicando el principio de diseño por responsabilidades para que cada clase tenga una sola razón de cambio – primer paso forma hacia el principio de Responsabilidad Única (SRP) que se estudiará en la semana 9.

1.1. Objetivos Específicos

- Comprender la diferencia entre encapsulamiento superficial (solo ocultar datos) y encapsulamiento profundo (proteger invariantes de clase).
- Implementar invariantes de clase mediante validaciones en setters y en el constructor.
- Aplicar el decorador @property en Python para crear atributos en solo lectura y de lectura / escritura con validación.
- Implementar getters/setters validados en C++17 con lanzamiento correcto std::invalid_argument.
- Diseñar clases siguiendo el principio de “una sola responsabilidad” como guía de descomposición.
- Avanzar en el Proyecto Integrador: encapsular correctamente 3 clases definidas en la Semana 1.

2. Marco Teórico

2.1. Invariantes de Clase – El Contrato Interno

Un invariante de clase es una condición que debe ser verdadera en todo momento en el que el objeto sea visible externamente, antes de cualquier método público y después de él. Los invariantes son la razón fundamental del encapsulamiento: si los atributos son públicos, nada impide que el código externo rompa el invariante (Meyer, 1997);

```
1  # Sin invariante — el objeto puede quedar en estado inválido
2  class Calificacion:
3      def __init__(self, valor):
```

```
        self.valor = valor  # PUBLICO — cualquiera puede escribir -
5 o 999


# Con invariante protegido — el objeto SIEMPRE es consistente
class Calificacion:
    _MIN, _MAX = 0.0, 20.0

    def __init__(self, valor: float):
        self._valor = None  # inicializar ANTES de llamar al setter
        self.valor = valor  # usa el setter con validación

    @property
    def valor(self) -> float:
        return self._valor

    @valor.setter
    def valor(self, v: float) -> None:
        if not (self._MIN <= v <= self._MAX):
            raise ValueError(
                f"Calificacion {v} fuera de rango
[{self._MIN},{self._MAX}]"
            )
        self._valor = v

    # Invariante: self._valor siempre pertenece a [0.0, 20.0]
```

2.2. Name Mangling en Python – _atributo

Python Implementa el ocultamiento mediante convención (_attr = protegido) y Mangling (__attr = privado). El doble guion bajo hace que Python renombre el atributo a _NombreClase__attr, dificultando el acceso accidental desde el exterior, aunque no lo impide completamente:

```
class Docente:
    def __init__(self, dni: str):
        self.__dni = dni  # almacenado como _Docente__dni


# Ejemplo de uso
d = Docente("12345678")

# Acceso directo al atributo privado → genera AttributeError
print(d.__dni)
# AttributeError: no existe __dni

# Acceso interno con nombre mangling → posible, pero no recomendado
print(d._Docente__dni)
# "12345678" — accesible, pero no deberías

# Acceso correcto: mediante @property pública
class Docente:
    def __init__(self, dni: str):
```

```
        self.__dni = dni

    @property
    def dni(self):
        return self.__dni
```

2.3. Propiedades Calculadas (read-only) en Python

No todo @property necesitar un setter. Una propiedad calculada (derivada de otros atributos) debe ser de solo lectura — intentar asignarla directamente debe resultar en AttributeError:

```
Estudiante:
    def __init__(self, nombre: str, apellidos: str):
        self._nombre = nombre
        self._apellidos = apellidos

    @property
    def nombre_completo(self) -> str:
        """Propiedad CALCULADA — sin setter, de solo lectura."""
        return f"{self._apellidos}, {self._nombre}"

    @property
    def iniciales(self) -> str:
        return f"{self._nombre[0]}.{self._apellidos[0]}."


# Ejemplo de uso
e = Estudiante("Juan", "Mamani Quispe")

print(e.nombre_completo)    # "Mamani Quispe, Juan"

# Intento de asignación → genera AttributeError porque no hay setter
e.nombre_completo = "otro"
# AttributeError: can't set attribute
```

2.4. Diseño de responsabilidades — ¿A quién pertenece este dato?

Una técnica de diseño fundamental es preguntarse: "¿Que clase tiene la responsabilidad de conocer X o hacer Y?" La respuesta guía de ubicación de atributos y métodos. Una clase mal diseñada tiene atributos que "no le pertenecen" o métodos que trabajan principalmente sobre datos de otra clase (Meyer, 1997):

Pregunta de diseño	Clase Responsable

¿Quién conoce el PPA de un estudiante?	Estudiante – porque tiene sus calificaciones
¿Quién sabe si un libro está disponible?	Libro – porque tiene su propio estado (disponible/prestado)
¿Quién formatea el nombre para imprimir?	No el docente — una clase Formateador o __str__separado.

3. Materiales

Herramientas	Uso
Python 3.12/Colab	Parte 1: propiedades, invariantes, name Mangling
g++ -std=c++17 -Wall	Parte 2: setters validados, const correctness
Draw.io/ StarUML	Diagramas de clases actualizado del proyecto (Act.5)
Git/Github	Commit de avance de la Semana 2 al repositorio del proyecto

4. Instrucciones Previas

1. Abrir notebook de Colab de la semana 1 y agregar una nueva sección “Semana 2”.
2. Tener el código de las clases refactorizadas de la Semana 1 disponible como base para esta práctica.
3. Para C++: compilar siempre con g++ -std=c++17 -Wall -Wextra -o prog main.cpp
4. Al terminar, hacer commit de todo avance al repositorio GitHub del proyecto con el mensaje “semana 2: encapsulamiento avanzado e invariantes”.

5. Desarrollo de actividad

5.1. Actividad 1 — @property avanzado — Invariantes con setters validados (Python)

Implementa la clase Curso del sistema académico con encapsulamiento estricto. El Curso tiene: código (string de formato SIS + 3 dígitos), nombre, créditos (1-6) y cupos_maximo (1-50). Todas las validaciones van en los setters correspondientes.

```python
import re

class Curso:
    _PATRON_CODIGO = re.compile(r"^SIS\d{3}$")
    _MIN_CREDITOS, _MAX_CREDITOS = 1, 6
    _MIN_CUPOS, _MAX_CUPOS = 1, 50

    def __init__(self, codigo: str, nombre: str, creditos: int, cupos_maximo: int):
        self.codigo = codigo        # usa setter
        self._nombre = nombre
        self.creditos = creditos    # usa setter
        self.cupos_maximo = cupos_maximo  # usa setter
        self.__inscritos = 0        # privado — no tiene setter público

    # --- Código ---
    @property
    def codigo(self) -> str:
        return self.__codigo

    @codigo.setter
    def codigo(self, valor: str) -> None:
        if not self._PATRON_CODIGO.match(valor):
            raise ValueError(f"Código inválido: {valor}. Debe tener formato SIS###")
        self.__codigo = valor

    # --- Créditos ---
    @property
    def creditos(self) -> int:
        return self.__creditos

    @creditos.setter
    def creditos(self, valor: int) -> None:
        if not (self._MIN_CREDITOS <= valor <= self._MAX_CREDITOS):
            raise ValueError(f"Créditos {valor} fuera de rango
[{self._MIN_CREDITOS}, {self._MAX_CREDITOS}]")
        self.__creditos = valor

    # --- Cupos máximo ---
    @property
    def cupos_maximo(self) -> int:
```

```python
        return self.__cupos_maximo

    @cupos_maximo.setter
    def cupos_maximo(self, valor: int) -> None:
        if not (self._MIN_CUPOS <= valor <= self._MAX_CUPOS):
            raise ValueError(f"Cupos {valor} fuera de rango [{self._MIN_CUPOS},
{self._MAX_CUPOS}]")
        self.__cupos_maximo = valor

    # --- Inscritos ---
    @property
    def inscritos(self) -> int:
        """Solo lectura — no tiene setter público."""
        return self.__inscritos

    @property
    def cupos_disponibles(self) -> int:
        """Propiedad calculada — sin setter."""
        return self.__cupos_maximo - self.__inscritos

    @property
    def esta_lleno(self) -> bool:
        """Propiedad calculada booleana."""
        return self.__inscritos >= self.__cupos_maximo

    def inscribir_estudiante(self) -> None:
        """Controla el invariante: inscritos <= cupos_maximo."""
        if self.esta_lleno:
            raise RuntimeError(f"Curso {self.__codigo} sin cupos disponibles")
        self.__inscritos += 1

    def __str__(self) -> str:
        return (
            f"{self.__codigo} - {self._nombre} "
            f"({self.__creditos} cr) "
            f"[{self.__inscritos}/{self.__cupos_maximo} inscritos]"
        )


# --- Pruebas -----------------------------------------------------
if __name__ == "__main__":
    c = Curso("SIS206", "POO II", 4, 30)
    print(c)
    print(f"Cupos disponibles: {c.cupos_disponibles}")

    for _ in range(3):
        c.inscribir_estudiante()
    print(c)

    # Invariantes — deben lanzar ValueError
    for caso in [
        ("X9", "nombre", 4, 30),      # código inválido
        ("SIS206", "nombre", 0, 30), # créditos fuera de rango
        ("SIS206", "nombre", 4, 0)    # cupos fuera de rango
    ]:
        try:
            Curso(*caso)
```

```
        except ValueError as ex:
            print(f"Error esperado: {ex}")
```

5.2. Actividad 2 — Propiedades calculadas y encadenamiento (Python)

Implemente la clase Docente que utiliza Curso mediante composición (una lista de recursos asignados). Las propiedades calculadas deben derivar su valor del estado actual del objeto, no almacenarse como atributos adicionales.

```
from datetime import date
import re

class Curso:
    _PATRON_CODIGO = re.compile(r"^SIS\d{3}$")
    _MIN_CREDITOS, _MAX_CREDITOS = 1, 6
    _MIN_CUPOS, _MAX_CUPOS = 1, 50

    def __init__(self, codigo: str, nombre: str, creditos: int, cupos_maximo: int):
        self.codigo = codigo
        self._nombre = nombre
        self.creditos = creditos
        self.cupos_maximo = cupos_maximo
        self.__inscritos = 0

    @property
    def codigo(self) -> str:
        return self.__codigo

    @codigo.setter
    def codigo(self, valor: str) -> None:
        if not self._PATRON_CODIGO.match(valor):
            raise ValueError(f"Código inválido: {valor}. Debe tener formato
SIS###")
        self.__codigo = valor

    @property
    def creditos(self) -> int:
        return self.__creditos

    @creditos.setter
    def creditos(self, valor: int) -> None:
        if not (self._MIN_CREDITOS <= valor <= self._MAX_CREDITOS):
            raise ValueError(f"Créditos {valor} fuera de rango
[{self._MIN_CREDITOS}, {self._MAX_CREDITOS}]")
        self.__creditos = valor

    @property
    def cupos_maximo(self) -> int:
        return self.__cupos_maximo

    @cupos_maximo.setter
```

```
    def cupos_maximo(self, valor: int) -> None:
        if not (self._MIN_CUPOS <= valor <= self._MAX_CUPOS):
            raise ValueError(f"Cupos {valor} fuera de rango [{self._MIN_CUPOS},
{self._MAX_CUPOS}]")
        self.__cupos_maximo = valor

    @property
    def inscritos(self) -> int:
        return self.__inscritos

    def inscribir_estudiante(self) -> None:
        if self.__inscritos >= self.__cupos_maximo:
            raise RuntimeError(f"Curso {self.__codigo} sin cupos disponibles")
        self.__inscritos += 1

    def __str__(self) -> str:
        return (f"{self.__codigo} - {self._nombre} "
                f"({self.__creditos} cr) "
                f"[{self.__inscritos}/{self.__cupos_maximo} inscritos]")


class Docente:
    _MAX_CURSOS = 3

    def __init__(self, dni: str, nombres: str, apellidos: str,
                 correo: str, fecha_nac: date):
        self.__dni = self._validar_dni(dni)
        self.__nombres = nombres
        self.__apellidos = apellidos
        self.__correo = self._validar_correo(correo)
        self.__fecha_nac = fecha_nac
        self.__cursos: list[Curso] = []

    # --- Validaciones ---
    @staticmethod
    def _validar_dni(dni: str) -> str:
        if not dni.isdigit() or len(dni) != 8:
            raise ValueError("El DNI debe tener exactamente 8 dígitos")
        return dni

    @staticmethod
    def _validar_correo(correo: str) -> str:
        if not correo.endswith("@unap.edu.pe"):
            raise ValueError("El correo debe terminar en @unap.edu.pe")
        return correo

    # --- Propiedades calculadas ---
    @property
    def nombre_completo(self) -> str:
        return f"{self.__apellidos}, {self.__nombres}"

    @property
    def edad(self) -> int:
        hoy = date.today()
        anios = hoy.year - self.__fecha_nac.year
        if (hoy.month, hoy.day) < (self.__fecha_nac.month, self.__fecha_nac.day):
            anios -= 1
```

```python
        return anios

    @property
    def total_creditos(self) -> int:
        return sum(c.creditos for c in self.__cursos)

    @property
    def total_inscritos(self) -> int:
        return sum(c.inscritos for c in self.__cursos)

    # --- Métodos ---
    def asignar_curso(self, curso: Curso) -> None:
        if len(self.__cursos) >= self._MAX_CURSOS:
            raise RuntimeError(f"{self.nombre_completo} ya tiene {self._MAX_CURSOS}
cursos asignados")
        self.__cursos.append(curso)

    def __str__(self) -> str:
        return (f"Dr. {self.nombre_completo} | Edad: {self.edad} "
                f"| Cursos: {len(self.__cursos)} | Total créditos:
{self.total_creditos}")


# --- Pruebas -------------------------------------------------------
if __name__ == "__main__":
    doc = Docente("12345678", "Aldo Hernan", "Zanabria Galvez",
                  "azanabria@unap.edu.pe", date(1980, 5, 14))

    poo2 = Curso("SIS206", "POO II", 4, 30)
    poo2.inscribir_estudiante()
    poo2.inscribir_estudiante()

    doc.asignar_curso(poo2)

    print(doc)
    print(f"Total inscritos en sus cursos: {doc.total_inscritos}")
```

5.3. Actividad 3 — Encapsulamiento en C++17 — const correctness y setters validados

Reimplemente la clase Curso en C++17 con: (i) atributos privados con convención de sufijo guion bajo, (ii) setters que lanzan std::invalid_argument, (iii) métodos constantes (const) para todos los getters y métodos de solo lectura.

```cpp
// Curso.hpp — Práctica S2, Actividad 3
#pragma once

#include <string>
#include <regex>
#include <stdexcept>
#include <iostream>

class Curso {
private:
    std::string codigo_;
    std::string nombre_;
    int creditos_;
    int cuposMaximo_;
    int inscritos_ = 0; // inicializado en declaración

public:
    // --- Constructor -----------------------------------------------
    Curso(const std::string& codigo, const std::string& nombre,
          int creditos, int cuposMaximo)
        : nombre_(nombre) {
        setCodigo(codigo);
        setCreditos(creditos);
        setCuposMaximo(cuposMaximo);
    }

    // --- Setters con validación ------------------------------------
    void setCodigo(const std::string& cod) {
        static const std::regex patron("^SIS[0-9]{3}$");
        if (!std::regex_match(cod, patron)) {
            throw std::invalid_argument("Código inválido: debe tener formato
SIS###");
        }
        codigo_ = cod;
    }

    void setCreditos(int c) {
        if (c < 1 || c > 6) {
            throw std::invalid_argument("Créditos fuera de rango [1,6]");
        }
        creditos_ = c;
    }

    void setCuposMaximo(int q) {
        if (q < 1 || q > 50) {
```

```
            throw std::invalid_argument("Cupos fuera de rango [1,50]");
        }
        cuposMaximo_ = q;
    }

    // --- Getters (const — no modifican el objeto) --------------------
    const std::string& getCodigo() const { return codigo_; }
    const std::string& getNombre() const { return nombre_; }
    int getCreditos() const { return creditos_; }
    int getCuposMaximo() const { return cuposMaximo_; }
    int getInscritos() const { return inscritos_; }

    // --- Propiedades calculadas (const) ------------------------------
    int cuposDisponibles() const {
        return cuposMaximo_ - inscritos_;
    }

    bool estaLleno() const {
        return inscritos_ >= cuposMaximo_;
    }

    // --- Lógica de negocio -------------------------------------------
    void inscribirEstudiante() {
        if (estaLleno()) {
            throw std::runtime_error("Curso sin cupos disponibles");
        }
        ++inscritos_;
    }

    // --- Representación en string ------------------------------------
    friend std::ostream& operator<<(std::ostream& os, const Curso& c) {
        os << c.codigo_ << " - " << c.nombre_
           << " (" << c.creditos_ << " cr)"
           << " [" << c.inscritos_ << "/" << c.cuposMaximo_ << " inscritos]";
        return os;
    }
};
```

5.4. Actividad 4 — Comparación Python vs C++ — tabla de equivalencias de encapsulamiento

Complete la tabla de equivalencias entre los mecanismos de encapsulamiento de Python y C++17. Luego responda la pregunta de análisis al pie.

Concepto	**tipo attr_ con acceso private:**	C++17
Atributo privado	__attr (name mangling) → accesible como _Clase__attr	tipo attr_ con acceso private:
Atributo protegido	_attr (convención)	tipo attr_ con acceso protected:

Getter de solo lectura	@property def attr(self)	const TipoR& getAttr() const
Getter + setter validado	@property def attr(self) + @attr.setter def attr(self, v)	TipoR getAttr() const + void setAttr(TipoR v) con validación
Propiedad calculada (R/O)	Método con @property sin setter	Método con @property sin setter
Método que NO modifica	Decorador @property o métodos que no cambian estado	Declarar método como const
Lanzar excepción de validación	raise ValueError("mensaje")	throw std::invalid_argument("mensaje")

Pregunta de análisis: En C++, un const Curso no puede llamar a inscribirEstudiante() aunque sea un

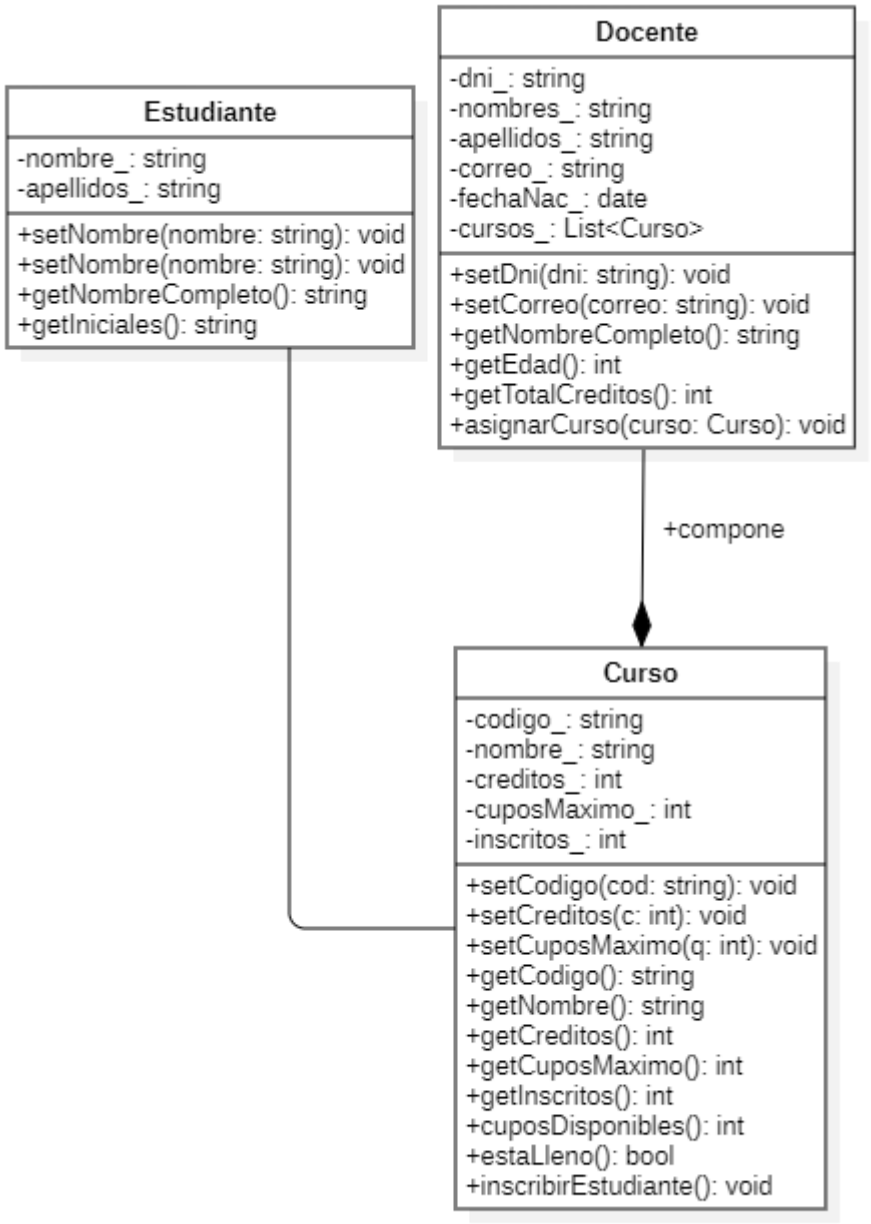
método perfectamente válido en tiempo de ejecución. ¿Por qué el sistema de tipos de C++ impone esta

restricción, y cuál es el beneficio de diseño que esto aporta?

En C++ cuando declaras un objeto como const Curso, el compilador interpreta que ese objeto no puede modificar su estado interno en absoluto. Por eso, solo permite invocar métodos marcados como const, que son aquellos que garantizan no alteran atributos,

El método inscribirEstudiante() incrementa el contador de inscritos, lo cual cambia el estado del objeto, y por lo tanto no puede ser llamado sobre un objeto constante. Aunque en tiempo de ejecución la operación sea válida, el sistema de tipos lo prohíbe porque rompe la promesa de inmutabilidad.

6. Diagramas UML



7. Preguntas de Reflexión

7.1. Explique la diferencia entre un atributo simplemente privado (que se puede modificar sin restricción desde dentro de la clase) y un atributo con invariante (que siempre valida antes de modificarse). ¿Cuándo podría saltarse la validación justificadamente?

Un atributo privado simple es aquel que está oculto al exterior pero puede modificarse libremente dentro de la clase. No hay ninguna restricción: cualquier método interno puede asignarle un nuevo valor.

En cambio, un atributo con invariante siempre pasa por un mecanismo de validación antes de cambiarse. Esto asegura que el objeto nunca entre en un estado inválido (por ejemplo, que los créditos de un curso estén siempre entre 1 y 6).

Saltarse la validación puede justificarse en situaciones controladas, como la inicialización en el constructor, donde se asigna un valor seguro conocido, o en operaciones internas que ya garantizan la consistencia.

7.2. En Python, el decorador @property permite que el cliente use e.creditos = 4 (sintaxis de asignación directa) aunque internamente ejecute un setter con validación. ¿Qué ventaja de interfaz aporta esto frente al enfoque C++ de llamar setCreditos(4) explícitamente?

En Python, el decorador @property permite que el cliente use una sintaxis natural de atributo (e.creditos = 4), aunque internamente se ejecute un setter con validación. Esto aporta una interfaz más clara y uniforme: el usuario del objeto no necesita saber si el atributo es calculado, validado o simple, siempre lo manipula igual.

En C++ el cliente debe llamar explícitamente a setCreditos(4), lo que hace evidente que se está invocando un método. La ventaja de Python es la transparencia y la legibilidad de la interfaz, que se siente más intuitiva.

7.3. La propiedad cupos_disponibles de la Actividad 1 se calcula cada vez que se invoca. ¿Cuándo sería justificado almacenarla como atributo adicional en lugar de calcularla? ¿Qué riesgo introduce almacenarla?

La propiedad cupos_disponibles se calcula cada vez que se invoca, restando cupos_maximo - inscritos. Esto es eficiente porque el cálculo es trivial.

Podría justificarse almacenarla como atributo adicional si el cálculo fuera muy costoso (por ejemplo, si dependiera de recorrer una lista grande o consultar una base de datos) y se necesitara acceder a ella muchas veces.

El riesgo de almacenarla es la desincronización: si alguien modifica inscritos o cupos_maximo pero olvida actualizar cupos_disponibles, el objeto quedaría en un estado inconsistente. Por eso, en la mayoría de casos es más seguro calcularla dinámicamente.

7.4. ¿Por qué marcar los getters como const en C++ es buena práctica incluso cuando "no parece importar"? Dé un ejemplo de la Actividad 3 donde omitir const causaría un error de compilación real.

Marcar los getters como const en C++ es buena práctica porque garantiza que no modifican el estado del objeto. Esto permite usar objetos constantes con seguridad y habilita optimizaciones del compilador.

Ejemplo:

```
1 const Curso c("SIS206", "POO II", 4, 30);
2 std::cout << c.getCreditos(); // funciona si getCreditos() es const
```

Si getCreditos() no estuviera marcado como const, el compilador daría error porque intentas llamar un método potencialmente modificador sobre un objeto constante. Así, el uso de const en getters refuerza la inmutabilidad y evita errores de compilación reales.